

PhysCC: A Domain-Specific Compiler for High-Performance Physics Simulations

Independent Research Report
Technical Portfolio for Academic Inquiry

February 2026

Abstract

This report presents **PhysCC**, a domain-specific compiler (DSL) designed to bridge the gap between abstract mathematical notation and hardware-optimized execution. By implementing a specialized multi-stage pipeline—from Unicode-aware lexical analysis to AVX2 SIMD code generation—PhysCC enables researchers to execute complex simulations, such as the Time-Dependent Schrödinger Equation (TDSE), at near-peak hardware performance (up to 1.46 GUPS). This research focuses on optimizing memory layouts (SoA) and core affinity on hybrid architectures to resolve the “Two-Language Problem” in scientific computing.

1 Introduction

Scientific computing historically suffers from a performance-productivity gap. High-level environments like Python offer rapid prototyping but lack the execution speed required for large-scale grids. Conversely, manual implementations in C++ or CUDA require extensive knowledge of hardware intrinsics. PhysCC solves this by allowing a declarative physics input that is lowered through an Intermediate Representation (IR) into architecture-specific machine code, specifically targeting Intel AVX2 and distributed MPI environments.

2 The PhysCC Compilation Pipeline

The architecture of PhysCC follows a modular “lowering” strategy, ensuring that physical semantics are preserved until the final instruction generation.

2.1 Lexical Analysis and Parsing

The Lexer utilizes a finite state machine to support the “Physics Lexicon,” identifying symbols such as ∇^2 and $\hbar$. The Recursive Descent Parser then constructs an Abstract Syntax Tree (AST), enforcing mathematical operator precedence and identifying the specific physical regime (Classical vs. Quantum).

2.2 IntegratorIR: Geometry and Stencil Lowering

The AST is lowered into a mid-level **IntegratorIR**. Here, symbolic operators are discretized into finite-difference stencils. For example, the Laplacian ∇^2 is expanded into a relative offset map $\{idx, idx \pm 1, idx \pm N\}$, mapping abstract math to linear memory addresses.

3 Technical Specification: Complex-Regime Optimization

The simulation of the Schrödinger Equation requires handling complex numbers ($\psi = \text{Re} + i\text{Im}$). Standard interleaved memory (AoS) results in poor SIMD utilization due to data shuffling overhead.

3.1 De-interleaved Structure of Arrays (SoA)

PhysCC automatically generates a **Structure of Arrays (SoA)** layout, separating real and imaginary components into distinct memory buffers:

$$\psi_{SoA} = \{[R_0, R_1, \dots, R_n], [I_0, I_1, \dots, I_n]\} \quad (1)$$

By loading 8 real and 8 imaginary points into separate 256-bit YMM registers using `_mm256_loadu_ps`, the backend achieves zero-shuffle arithmetic, saturating the execution ports of the CPU.

4 Benchmarking and Performance Analysis

Testing was conducted on an Intel i5-1235U (2P+8E cores) to evaluate throughput in Giga-Updates Per Second (GUPS).

Regime	Optimization	Throughput (GUPS)	Hardware Constraint
Classical Wave	AVX2 + P-Core Bind	1.467	Compute Bound
Quantum (2D)	De-interleaved SoA	0.559	L3 Cache/Bandwidth
Quantum (3D)	7-Point Stencil	0.210	DDR4 Memory Wall

Table 1: PhysCC Throughput Metrics

The results indicate that while the classical path is compute-bound, the quantum path effectively tests the memory subsystem limits due to the additional Potential (V) grid overhead.

5 Data Flow and Scientific Observation

Final simulation data is exported as a high-fidelity CSV. In the quantum regime, PhysCC performs a reduction to the observable **Probability Density**:

$$P(x, y) = |\psi(x, y)|^2 = \text{Re}(\psi)^2 + \text{Im}(\psi)^2 \quad (2)$$

Validation via Python visualization confirms the capture of wave packet dispersion and **Quantum Tunneling** through potential barriers, providing empirical proof of the compiler’s numerical accuracy.

6 Future Work and Conclusion

PhysCC demonstrates that the burden of hardware optimization can be effectively shifted from the researcher to the compiler. Future work includes implementing **Adaptive Mesh Refinement (AMR)** and automated GPU offloading via **SYCL**. This project, licensed under **GPLv3**, provides an open-source framework for high-performance, reproducible scientific discovery.